

SecureChat Project

Assignment 3 – COSC2639



Table of Contents

Table of Contents	2
Links.....	3
Summary.....	3
Introduction.....	4
Motivation	4
High-Level View	4
Application Details	4
Application Features	4
AWS Services	5
Beneficiaries.....	6
Related Works	6
System Architecture.....	7
Architecture Diagram.....	7
System Description	7
Elastic Beanstalk.....	7
S3	8
API Gateway	8
Lambda.....	8
DynamoDB	8
Event Bridge.....	8
Glue.....	9
Athena	9
Certificate Manager	9
References.....	10

Links

Live URL: <https://a3securechat-env.eba-2b5smmsi.us-east-1.elasticbeanstalk.com/>

Summary

SecureChat is to a simple private and secure messaging platform that aims to protect its user's data while maintaining its usability. In making privacy and security the application's focus, both the application's features and AWS services are utilised to facilitate these criteria. Furthermore, the application is hosted using a serverless architecture which allows for high scalability and provides the best possible performance as the application's needs increase without the need to redeployment of AWS services and respective resources.

Introduction

Motivation

The motivation for this project stems from the lack of focus on privacy and security when it comes to communicating online. Presently available applications that facilitate these needs are lacking in either privacy or security features, which users will have to choose between when thinking about when registering to use these services. Today's applications are also incentivized by profits, which means user data could be potentially shared without explicit knowledge by the user when using these platforms. My application 'SecureChat' endeavours to provide a secure messaging application that adheres to both private and secure practices.

High-Level View

Application Details

The application will be built using python and flask as the web framework. The frontend uses the bootstrap frontend toolkit [7] to provide user-friendly design elements for an overall pleasant user experience. AWS (Amazon Web Services) services are accessed through API (Application Program Interfaces) calls which are required to provide application functionality i.e., accessing user's account details, contact lists, conversation list and messages.

To increase security, all input data is sanitised to remove any malicious code that a bad actor may attempt to run in an input field. When uploading images, the file's names are checked for any malicious code that may be trying to be injected and if the file itself is one of three file formats: png, jpg, jpeg. All cookies are signed to prevent cookie tampering from occurring. All data in transport to Elastic Beanstalk utilise end-to-end encryption which incorporates HTTP to HTTPS redirection, meaning the traffic from the site is always sent securely. To increase privacy, details of users are only stored in one place and encrypted at rest within DynamoDB. Analytics is calculated by summarising table data and does not copy user's data to any other internal or external service. The files stored for analytics using the S3 service, including information stored in other buckets such as user profile images, are encrypted by an Amazon S3-managed key (SSE-S3) at rest.

Application Features

The application will have a login and register area that the user can use to first register an account with the system and then login to use the chat services. When the user logs in, they will be loaded into the conversations and messages page.

The top area will include a navbar where they can see the conversations and messages page, a contacts page to manage who they can converse with, their username that links to their profile and a logout link to logout of the current session. In the conversations and messages page, below the navigation bar will be a conversation list area on the left, and messages area on the right. On the left area just above the conversation list, an add conversation button exists to allow users to start a conversation from a list of their contacts. When a user clicks on a new or previously made conversation within the conversation list; the messages area will update to show all associated messages for that conversation. Messages sent by the user will always be shown on the left, messages from others will be shown on the right of the conversation window. From the contact page the user can view their contacts or add contacts via a unique code provided to them by the application. Users can also remove contacts they have added previously. A user can update their sign in details which include their username, password, and email address and additionally their profile image within their profile page. When they update their username, email or password, the user will be asked to log in again into their account to finalise the changes. An analytics dashboard page is also available but only for an admin account. The page will show certain analytics as provided by Athena i.e., number of registered users, number of conversations total, number of messages sent total, number of users, conversations, and messages over time as a graph.

AWS Services

The platform will use AWS's Elastic Beanstalk, API Gateway, Lambda, S3 (Super Simple Storage), DynamoDB, Event Bridge, Glue, Athena, and Certificate Manager services to facilitate a secure serverless chat application that can scale its resources according to usage parameters.

Elastic Beanstalk will host the application built in python and will encrypt traffic end to end meaning traffic from the client to the Elastic Beanstalk's instance is encrypted [2][12][14][15]. The application will use an API Gateway RESTful API to facilitate communication with DynamoDB via pre-created Lambda functions [4] that trigger based on the route that was accessed via the endpoint [3]. Endpoints are secured using a lambda authoriser function therefore only allowing the application to access the API. Routes will be created to provide application functionality and exporting summarised table data from DynamoDB to S3 Buckets [6]. DynamoDB will store user account information, each new chat between users, user's conversation and contact lists [5][9] and is encrypted at rest. S3 will store application source code, user media which is currently profile images and analytics and query data and results [6] and is also encrypted at rest.

Event Bridge will run a daily cron job which runs a lambda function that will copy summarised table data to specific folders within S3 [6] from DynamoDB to be then interpreted by Glue via a crawler [11]. This creates tables that can then be queried by the admin account from Athena in the form of SQL queries. This supplies analytics data to the admin account in the frontend [10][13]. Certificate Manager provides a self-signed SSL certificate for encryption purposes [15].

Beneficiaries

This application is suited most for end-users who wish to use a messaging platform that cares about their personal information and want to be able to communicate with their friends or peers in a secure and private way without worrying about who can see their data. While it is made to be secure, the platform would not necessarily be a good fit for a corporate setting, as it is a basic messaging application. While it could be adapted for a business setting, it does not possess features that other business-oriented applications provide that would prove more useful for everyday use.

Related Works

Below is a list of secure messaging platforms that are currently available. There may be other platforms that exist, but these are the most well-known:

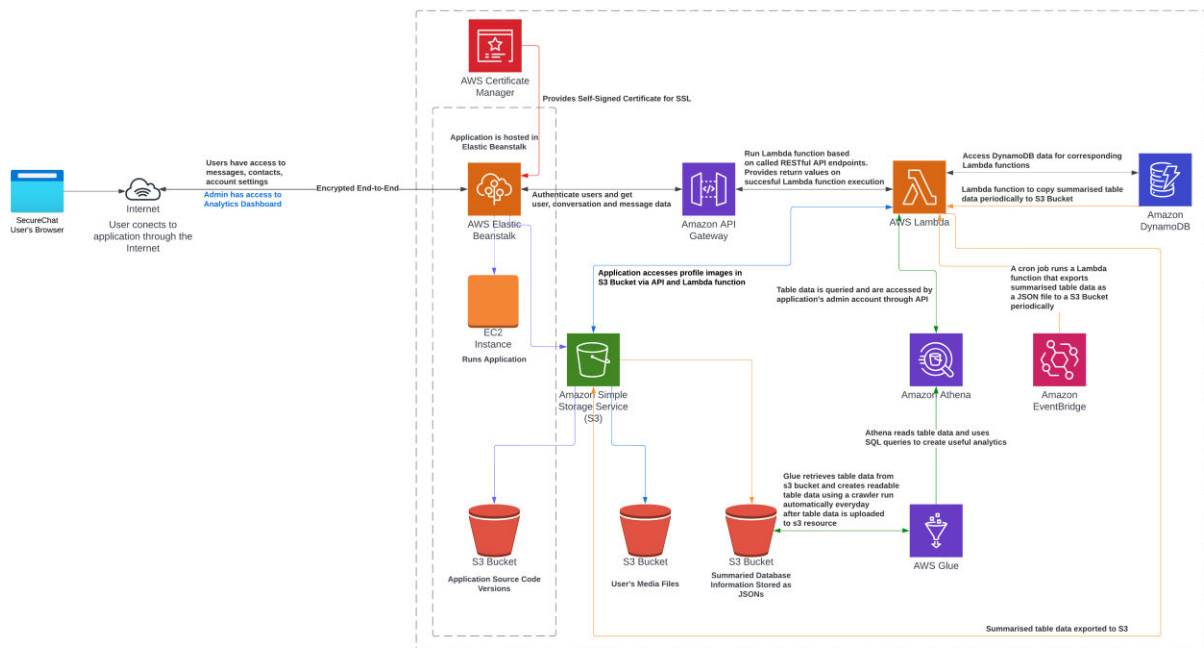
- Signal
- Telegram
- WhatsApp

Signal, Telegram and WhatsApp all utilise secure protocols to its users for communicating with each other. Technologies such as E2EE (end to end encryption) are used when sending and receiving messages between senders and recipients although the implementation of these technologies vary depending on the platform. For example, Signal and Telegram both use E2EE by default but WhatsApp only employs it for messaging between users but not between users and business accounts. Furthermore, Telegram uses its own E2EE protocols which are maintained by Telegram itself. This means that if desired, someone within Telegram who has access to the encryption keys could see message data. There is also the practice of how the platform stores user's account information and message data which again defer between the listed applications.

Signal is a not-for-profit organisation and therefore will not sell the data within their system as mentioned in their pledge. Telegram and WhatsApp are a for-profit platform as they are seeking to monetise their platform to a degree and therefore may rent or sell user information. WhatsApp is owned by Meta and have had issues with user data being processed by its parent company in the past. [1]

System Architecture

Architecture Diagram



System Description

Elastic Beanstalk

Elastic Beanstalk hosts the chat application and is built using python and uses flask as a web framework. An EC2 Instance is used to run the application and a S3 bucket to store source code. Elastic Beanstalk will be configured to use HTTPS termination at the instance with end-to-end encryption enabled for all application traffic. This is achieved by configuring the security groups for the application load balancer and the instance's load balancer.

After HTTPS termination is activated at the load balancer, the security groups that control HTTPS traffic between the load balancer and its instances are modified to allow HTTPS traffic and to decrypt and encrypt traffic that travels between them. Traffic is only sent via port 443 and only accepts a specific self-signed certificate from the domain and no other traffic. HTTP to HTTPS redirection is also enabled meaning all traffic will always be sent securely. [2][12][14]

S3

The S3 service provides three S3 buckets that are utilising by the application. The first bucket is used for storing the source code for the application by Elastic Beanstalk. The second bucket is for storing user media files, which currently is profile images, and the third bucket holds summarised table data from DynamoDB and query results from Athena. All files in buckets are encrypted with Amazon S3-managed key (SSE-S3) at rest [6]

API Gateway

The API Gateway uses the RESTful API that has generated routes that trigger specific Lambda functions. The API uses a lambda authoriser to authenticate who is accessing the API, only allowing the application to access it. [3]

Lambda

Specific Lambda functions are created and are connected to RESTful API routes to retrieve user details, conversation list, conversations, and message data. Also contains functions to export tables and their summarised data from DynamoDB to a S3 bucket as triggered by a cron job. [4]

DynamoDB

Contains tables for user account data and tables for each conversation present on the platform and each user's contact list. A cron job triggers a lambda function daily that calculates summarised table data to an S3 bucket to be used as a basis for querying and analytics by the application's admin account through Athena. All table data is encrypted at rest [5][9]

Event Bridge

Event Bridge triggers a cron job daily to export summarised DynamoDB table data into an S3 bucket to be used by Athena for analytics. The cron job activates a lambda function to accomplish this task. [8]

Glue

Glue will run a crawler that creates readable table data from the exported summarised DynamoDB table data saved in the S3 bucket to be used for Athena to query and create analytics to be shown in the admin account. [11]

Athena

Athena will use the table data available in the S3 bucket to run queries as required by the analytics dashboard that the admin has access to. This is accessed through the API Gateway through a RESTful API route connected to a lambda function. [10][13]

Certificate Manager

Certificate Manager provides a self-signed SSL certificate to provide SSL for the application. [15]

References

- [1] M. Eddy, "The best secure messaging apps for 2022," *PCMag Australia*, 23-May-2022. [Online]. Available: <https://au.pcmag.com/security/94184/the-best-secure-messaging-apps-for-2022>. [Accessed: 18-Oct-2022].
- [2] "Web app deployment - AWS (Amazon Web Services) elastic beanstalk - amazon web services," *Web app deployment*. [Online]. Available: <https://aws.amazon.com/elasticbeanstalk/>. [Accessed: 18-Oct-2022].
- [3] "Amazon API Gateway - Amazon Web Services," *Amazon API Gateway*. [Online]. Available: <https://aws.amazon.com/api-gateway/>. [Accessed: 18-Oct-2022].
- [4] "AWS Lambda - Amazon Web Services," *AWS Lambda*. [Online]. Available: <https://aws.amazon.com/lambda/>. [Accessed: 18-Oct-2022].
- [5] "DynamoDB: Everything you need to know about Amazon Web Service's NoSQL database," *DynamoDB*. [Online]. Available: <https://aws.amazon.com/dynamodb/>. [Accessed: 18-Oct-2022].
- [6] "Amazon S3 - Amazon Web Services," *S3*. [Online]. Available: <https://aws.amazon.com/s3/>. [Accessed: 18-Oct-2022].
- [7] B. Bootstrap, "Bootstrap Documentation," *Bootstrap v5.2*, 2022. [Online]. Available: <https://getbootstrap.com/docs/5.2/getting-started/introduction/>. [Accessed: 18-Oct-2022].
- [8] "Serverless event router - Amazon Event Bridge - Amazon Web Services," *Amazon Event Bridge*, 2022. [Online]. Available: <https://aws.amazon.com/eventbridge/>. [Accessed: 21-Oct-2022].
- [9] J. Beswick, "Understanding database options for your serverless web applications - AWS," *Amazon*, 03-Aug-2020. [Online]. Available: <https://aws.amazon.com/blogs/compute/understanding-database-options-for-your-serverless-web-applications/>. [Accessed: 21-Oct-2022].
- [10] R. Deshapriya, "Connecting to AWS Athena databases using Python," *Medium*, 04-Oct-2022. [Online]. Available: <https://medium.com/codex/connecting-to-aws-athena-databases-using-python-4a9194427638>. [Accessed: 21-Oct-2022].
- [11] A. Web Services, "Glue," *Glue - Boto3 Docs 1.26.8 documentation*, 2022. [Online]. Available:

<https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/glue.html>. [Accessed: 29-Oct-2022].

- [12] A. Web Services, "Deploying a flask application to Elastic Beanstalk," Amazon Web Services, 2022. [Online]. Available: <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create-deploy-python-flask.html>. [Accessed: 30-Nov-2022].
- [13] A. Web Services, "Athena," Athena - Boto3 Docs 1.26.8 documentation, 2022. [Online]. Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/athena.html>. [Accessed: 28-Oct-2022].
- [14] A. Web Services, "Configuring end-to-end encryption in a load-balanced elastic beanstalk," Amazon Web Services, 2022. [Online]. Available: <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/configuring-https-endtoend.html>. [Accessed: 08-Nov-2022].
- [15] A. Web Services, "What Is AWS Certificate Manager?" Amazon Web Services, 2022. [Online]. Available: <https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html>. [Accessed: 16-Nov-2022].